

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**МНОГООБЛАЧНО РАЗГРЪЩАНЕ НА
ИЗЧИСЛИТЕЛНО ИНТЕНЗИВНА УСЛУГА ОТ
ЕДНО ДЕКЛАРАТИВНО ОПИСАНИЕ НА
ИНФРАСТРУКТУРАТА И СРАВНИТЕЛНА
ОЦЕНКА НА ДВА ОБЛАЧНИ ДОСТАВЧИКА**

КУРСОВ ПРОЕКТ

по дисциплина „Облачни изчисления и технологии“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р Антония Ташева

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД .	3
1.1. Сигурност при работа с облачни технологии	4
1.2. Изводи от прегледа	5
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Начин на изпълнение на изчислителния работник	6
2.2. Инструмент за декларативно описание на инфраструктурата	7
2.3. Генератор на синтетично натоварване	7
2.4. Събиране на метрики	8
2.5. Обобщение на направените избори	9
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	10
3.1. Изпълними файлове и потоци от данни	10
3.2. Единица работа	11
3.3. Метрика, която управлява мащабирането	12
3.4. Политика за мащабиране	12
3.5. Договор на модулите за инфраструктура	13
3.6. Съпоставяне на изчислителния ресурс	13
3.7. Модел на идентичността и достъпа	13
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	15
4.1. Изчислителното ядро	15
4.2. Работникът	15
4.3. Преносимост на входно изходния слой	16
4.4. Обратният посредник и обединяването на метрики	16
4.5. Контролерът за мащабиране	17
4.6. Генераторът на натоварване	17
4.7. Моделът на разхода	17
4.8. Модулни тестове	18
4.9. Контейнеризация	18

4.10. Описание на инфраструктурата	19
4.11. Мерки срещу изтичане на данни за достъп	19
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНО СРАВНЕНИЕ	20
5.1. Обхват на действително проведените измервания	20
5.2. Конфигурация на машината	20
5.3. Параметри на натоварването	21
5.4. Условия за валидност на едно измерване	21
5.5. Проведени експерименти	22
5.6. Отчитани величини	22
5.7. Известни ограничения на методиката	23
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	24
6.1. Капацитет спрямо брой реплики	24
6.2. Време за реакция при увеличаване на набора	26
6.3. Поведение на контролера под натоварване	26
6.4. Размер на контейнерния образ	28
6.5. Цена за единица извършена работа	28
6.6. Сравнение между двамата доставчици	29
6.7. Обобщен анализ	29
ЗАКЛЮЧЕНИЕ	31
ЛИТЕРАТУРА	33
ПРИЛОЖЕНИЯ	35

УВОД

Публичните облачни платформи се описват в маркетинговите си материали като взаимозаменяеми: една и съща услуга може да бъде разгърната при кой да е доставчик, а изборът между тях се свежда до цена. На практика взаимозаменяемостта е ограничена. Всеки доставчик предлага собствен набор от управлявани услуги, собствен модел на идентичността, собствени единици за таксуване и собствено поведение при автоматично мащабиране. Твърдението, че две среди са еквивалентни, обикновено се приема без проверка, защото проверката изисква едно и също натоварване да бъде разгърнато и измерено на две места при съпоставими условия.

Настоящият курсов проект изгражда *Stratus*, изчислително интензивна услуга, опакована в контейнер и разгръщана едновременно в Amazon Web Services и Microsoft Azure от едно декларативно описание на инфраструктурата. Услугата приема заявки за изчисление, изнася собствени метрики и се мащабира хоризонтално под синтетично натоварване, като решението за мащабиране се взема от контролер, реализиран в проекта. Двете разгръщания се различават единствено по стойността на един параметър, което прави сравнението между тях осмислено: измерва се различието между доставчиците, а не различието между две ръчно настроени системи.

Целта на проекта е да провери до каква степен едно декларативно описание на инфраструктурата е достатъчно за разгръщането на една и съща услуга при два независими доставчика, и да измери разликата между тях по закъснение, поведение при мащабиране и цена за единица извършена работа.

За постигането на целта са формулирани следните **задачи**:

1. преглед на фундаменталните понятия в облачните изчисления, на моделите на обслужване IaaS, PaaS и SaaS, на видовете виртуализация и на клъстерните и GRID технологиите, върху които стъпват съвременните платформи;
2. анализ на възможните решения за изчислителна услуга, за инструмент за декларативно описание на инфраструктурата и за генератор на натоварване, и обосновка на направения избор;
3. проектиране на архитектура, в която платформено зависимата част е сведена до отделни модули, а описанието на услугата остава общо за двата доставчика;
4. програмна реализация на изчислителния работник, на контейнера и на модулите за

инфраструктура, заедно със събирането на метрики;

5. изграждане на методика за сравнително измерване при съпоставими условия;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на проекта е ограничен до един тип изчислително натоварване, до два доставчика и до два региона в Европа. Извън обхвата остават хибридните и частните облаци, миграцията на състояние между доставчиците по време на работа, както и услугите за управление на бази от данни. Всички идентификатори на сметки, абонаменти и ключове за достъп остават извън хранилището с изходния текст и се подават чрез променливи на средата.

Докъде е стигнала работата. Този абзац стои в увода, а не по-нататък, защото определя как да се четат всички числа в документа. Изградени и измерени са услугата, слой за наблюдение, контролерът за мащабиране и генераторът на натоварване, като измерванията са направени върху локален стек от контейнери. Описанието на инфраструктурата за двамата доставчици е написано, но **не е прилагано**: няма данни за достъп до сметка при нито един от тях и terraform не е инсталиран на използваната машина. Затова в документа няма нито едно измерване при AWS или Azure, а местата, които изискват такова, са явно означени като незапълнени. Обхватът на действително проведените експерименти е описан в Раздел 5.1.

Структура на изложението. Раздел I разглежда предметната област и литературните източници. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката за измерване, а Раздел VI получените резултати и техния анализ.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Понятието *облачни изчисления* има работно определение, върху което стъпва по-голямата част от литературата и на което се позовават самите доставчици. То описва модел за достъп при поискване до споделен резерв от изчислителни ресурси, които се предоставят и връщат обратно бързо и с минимално административно взаимодействие, и извежда пет съществени характеристики: самообслужване при поискване, широк мрежов достъп, обединяване на ресурсите, бърза еластичност и измерена услуга [1]. Последните две са пряко относими към този проект: еластичността е това, което се проверява при синтетичното натоварване, а измерената услуга е това, което прави сравнението по цена възможно изобщо.

Същият източник разграничава трите модела на обслужване. При *инфраструктура като услуга* (IaaS) потребителят получава изчислителни възли, мрежа и съхранение и сам отговаря за операционната система и всичко над нея. При *платформа като услуга* (PaaS) доставчикът поема средата за изпълнение, а потребителят предоставя само приложението. При *софтуер като услуга* (SaaS) потребителят използва готово приложение и не управлява нищо от инфраструктурата под него. Границата между IaaS и PaaS в съвременните управлявани услуги за контейнери е размита и точно тази размита граница е една от причините сравнението между двамата доставчици да не е тривиално: еднакво наименована услуга при двамата може да лежи от различни страни на границата. Ранният анализ на икономиката на облака [2] формулира и защо еластичността има стойност, надхвърляща средната утилизация: тя премахва нуждата от оразмеряване по върховото натоварване.

Технологичната основа на обединяването на ресурси е *виртуализацията*. Класическата хардуерна виртуализация чрез хипервайзор изолира пълни виртуални машини, всяка със собствено ядро, и е описана подробно за хипервайзора Xen [3]. Виртуализацията на ниво операционна система, известна като контейнеризация, споделя едно ядро между изолирани пространства от имена и постига значително по-кратко време за стартиране за сметка на по-слаба изолация [4]. Между двата подхода стоят леките виртуални машини, проектирани специално за многонаемни среди с кратко живеещи задачи [5]. Изборът между тези три степени на изолация определя както времето за

реакция при мащабиране, така и профила на риска при сигурността, и затова се разглежда в Раздел II.

Управлението на голям брой еднакви изчислителни възли води до *клъстерните технологии*. Индустриалният опит с планировчик за клъстери в мащаба на един голям доставчик е описан в [6], а изведените от него уроци и връзката им със следващото поколение системи за оркестрация са обобщени в [7]. Историческият предшественик на този клас системи са *GRID технологиите*, чиято цел е обединяването на ресурси между отделни административни организации и които въвеждат понятието виртуална организация [8]. Разликата между GRID и облака е преди всичко в собствеността и модела на таксуване, а не в техническата задача, и това е полезно за настоящия проект, защото показва, че управлението на разпределени ресурси е по-стара задача от търговските облачни платформи.

Съхранението в облака е отделен клас проблеми. Обектните хранилища жертват част от семантиката на файловата система в замяна на висока достъпност и практически неограничен обем. Два основополагащи текста описват вътрешното устройство на такива системи при два различни доставчика и правят различен избор по оста достъпност спрямо съгласуваност [9, 10]. За настоящия проект този избор има пряко следствие: ако входните и изходните данни на изчислителния работник се четат и пишат в обектно хранилище, поведението при едновременен достъп зависи от гаранциите на конкретния доставчик, а не от кода на работника.

1.1. Сигурност при работа с облачни технологии

Сигурността в облака се управлява по модела на споделената отговорност: доставчикът отговаря за сигурността на самата платформа, а потребителят за всичко, което разгръща върху нея. Практическото следствие за този проект е отказът от дълготрайни ключове за достъп в полза на *управлявана идентичност*, при която изчислителният възел получава краткосрочен маркер за достъп от самата платформа и никакъв секрет не се записва в хранилището с изходния текст. Точните имена на механизмите при двамата доставчици и разликите между тях се описват в Раздел III.

[TODO: Да се допълни: сравнителна таблица на моделите на идентичност при двамата доставчици, с препратка към официалната документация на всеки от тях.]

1.2. Изводи от прегледа

Прегледът показва, че фундаменталните понятия са общи за двете разглеждани платформи, докато конкретните механизми се различават. Това оправдава подхода на проекта: общото се изразява веднъж, в декларативно описание, а различното се изнася в отделни модули. Прегледът показва също, че липсва достъпно сравнение на двата доставчика при контролирано еднакво натоварване, което е приносът, който този проект се опитва да даде в рамките на своя ограничен обхват.

[TODO: Да се посочат конкретните публикувани сравнения между доставчици, ако при допълнителното търсене бъдат намерени такива, и да се уточни в какво настоящата постановка се различава от тях.]

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Изборите в този проект са четири: как се изпълнява изчислителният работник, как се описва инфраструктурата, как се създава натоварването и как се събират метриките. Всеки от тях се оценява по три критерия, общи за целия проект: наличие на съпоставим еквивалент при двамата доставчици, възможност описанието да остане едно, и пригодност на получените измервания за сравнение.

2.1. Начин на изпълнение на изчислителния работник

Разгледани са три възможности. Първата е *виртуални машини* с ръчно инсталирано приложение, тоест чист IaaS. Тя дава пълен контрол и почти еднакво поведение при двамата доставчици, но времето за стартиране на нов възел е от порядъка на минути, което прави наблюдението на еластичността бавно и скъпо. Втората е *управлявана услуга за контейнери* без достъп до възлите под нея. Тя намалява административната работа, но всеки доставчик я оформя различно и еднаквото описание става трудно. Третата е *Kubernetes* в управляван вид, тоест Amazon EKS и Azure AKS, при която описанието на самото приложение е идентично и различно остава само описанието на клъстера.

Първоначално беше избран третият вариант, тъй като при него описанието на работното натоварване буквално съвпада при двамата доставчици. При реализацията изборът беше **преразгледан в полза на втория**, тоест управлявана услуга за контейнери: AWS ECS с Fargate и Azure Container Apps.

Причината е емпирична. Построеният локален стек показва, че услугата е един контейнер зад обратен посредник, с контролер за мащабиране отвън, тоест не използва нищо от Kubernetes освен способността му да държи N реплики живи и достъпни на едно име. При това положение управляваният Kubernetes добавя постоянна такса за управляващ слой, описание на клъстера от няколкостотин реда при всеки доставчик, и второ ниво на мащабиране, тоест мащабиране на възлите под мащабирането на репликите, което замъглява точно величината, която проектът измерва. Услугите за контейнери без достъп до възлите съответстват по-точно на построената топология, а тяхното правило за мащабиране следи цел по същия начин като политиката, реализирана в проекта.

Цената на промяната е реална и се посочва: описанието на *работното натоварване*

вече не съвпада буквално при двамата доставчици. То е заменено с общ обект, който описва услугата, и с два модула, които го превеждат. Количествената последица от това е измерена и съобщена в Раздел VI.

2.2. Инструмент за декларативно описание на инфраструктурата

Скриптовете на командните интерфейси на самите доставчици отпадат веднага: те са императивни, не са идемпотентни и не позволяват общо описание. Собствените средства на доставчиците, AWS CloudFormation и Azure Bicep, са зрели, но всеки от тях работи само при своя доставчик, което би означавало две несвързани описания. Остават инструментите с множество доставчици: Terraform и неговото разклонение OpenTofu, и Pulumi, при който инфраструктурата се описва на език за общо предназначение.

Избран е Terraform, съответно OpenTofu. Причината е, че при него разликата между двата доставчика се локализира в отделни модули, а общата част, тоест променливите, изходите и редът на разгръщане, остава една. Pulumi предлага същото, но въвежда зависимост от среда за изпълнение на език, който иначе не участва в проекта. Разликата между Terraform и OpenTofu е лицензионна, не техническа, и изборът между тях не влияе на резултатите.

2.3. Генератор на синтетично натоварване

Изискването към генератора е едно: да произвежда еднакво натоварване срещу двете разгръщания и да отчита закъснението по персентили, а не само средна стойност. Разгледани са wrk, k6 и Apache JMeter. Първият е най-лек и дава най-малко собствено изкривяване, но описанието на сценария при него е ограничено. Третият е най-богат, но е тежък и изисква отделна инфраструктура за самия себе си.

Първоначално беше избран k6. При реализацията и този избор беше преразгледан, и генераторът е **написан в проекта**, на C++, като част от същото дърво на изграждане.

Причината е ограничение, поставено пред целия проект: изграждане с нула задължителни външни зависимости. Хранилището е публично и трябва да се изгражда на чиста машина само с компилатор на C++20 и CMake. k6 е отделен изпълним файл, който трябва да бъде инсталиран преди измерването да е възможно, тоест въвежда точно тази зависимост. Собственият генератор струва около 160 реда, използва вече наличния клиентски код за HTTP и добавя нещо, което е нужно на този експеримент: избор между

затворен и отворен контур. Вторият режим е този, при който забавена услуга натрупва време на чакане в отчетеното закъснение, вместо клиентът тихо да намали скоростта си, и е режимът, който показва за какво служи контролерът за мащабиране.

[TODO: При действително разгръщане да се потвърди, че генераторът се пуска от една и съща машина срещу двете разгръщания, и да се опише как се отчита разликата в мрежовото разстояние до двата региона.]

2.4. Събиране на метрики

Собствените средства за наблюдение на доставчиците, Amazon CloudWatch и Azure Monitor, са налични без допълнителна работа, но измерват различни величини по различен начин и затова не са годни за пряко сравнение. Затова метриките се събират от самото приложение в общ формат, а средствата на доставчика се използват само за контролна проверка.

Избран е форматът на Prometheus за текстово представяне на метрики, но не и самият Prometheus като задължителна зависимост. Приложението излага метриките в този формат, а събирането и обединяването им са реализирани в проекта, по същата причина, поради която е реализиран и генераторът: изграждане без външни зависимости. Изборът на формат, а не на програма, запазва същественото: всяка система за наблюдение, която разбира формата, включително самият Prometheus, може да чете измерванията без промяна в приложението. Grafana отпада заедно с това, тъй като табло не добавя измерване, а изглед към същите данни.

2.5. Обобщение на направените избори

Таблица 1. Направени избори, първоначални и след реализацията

Решение	Първоначален избор	Окончателен избор	Цена на промяната
Изпълнение на работника	Управляван Kubernetes, EKS и AKS	AWS ECS с Fargate и Azure Container Apps	Описанието на натоварването вече не съвпада буквално, а се превежда от два модула
Описание на инфраструктурата	Terraform или OpenTofu	Без промяна	Няма
Генератор на натоварване	k6	Собствен, на C++	Около 160 реда за поддръжка, срещу премахната външна зависимост
Събиране на метрики	Prometheus и Grafana	Форматът на Prometheus, със собствено събиране	Няма готово табло, метриките се четат от файлове и от крайната точка

Общото между трите промени е един и същ критерий, който не беше формулиран достатъчно ясно в началото: изграждане с нула задължителни външни зависимости. Той се оказва по-силен от удобството на готовите инструменти, и това е записано тук, а не премълчано, защото първоначалните обосновки по-горе остават четими и без него биха противоречили на реализацията.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е проектирана около едно ограничение: описанието на услугата трябва да е едно, а различията между двамата доставчици трябва да са изнесени в ясно очертани места, чийто брой може да бъде преброен. Ако различията се разпръснат из цялото описание, сравнението губи смисъл, защото вече не се сравняват две среди, а две различни системи.

Архитектурата има четири слоя. *Изчислителният работник* е приложение на C++20, което приема заявка по HTTP, изпълнява изчислително интензивна задача с известна сложност и връща резултата. Той не знае при кой доставчик работи: портът, размерът на пула от нишки и адресът на хранилището идват от променливи на средата. *Контейнерният образ* е един и същ за всички разгръщания и носи и петте изпълними файла на проекта. *Описанието на инфраструктурата* се състои от обща коренна конфигурация и два модула, по един за всеки доставчик, с еднакъв интерфейс от входни променливи и изходи. *Слоят за наблюдение и управление* събира метрики от работниците, обединява ги и взема решенията за мащабиране.

Разделението между общото и платформено зависимото минава точно по границата на модулите за инфраструктура. Общи остават: изчислителният работник, обратният посредник, генераторът на натоварване, контролерът за мащабиране, моделът на разхода, контейнерният образ и обектът, който описва услугата. Платформено зависими са: създаването на мрежата, създаването на изчислителната платформа, създаването на кофата или контейнера за обекти и обвързването на идентичността.

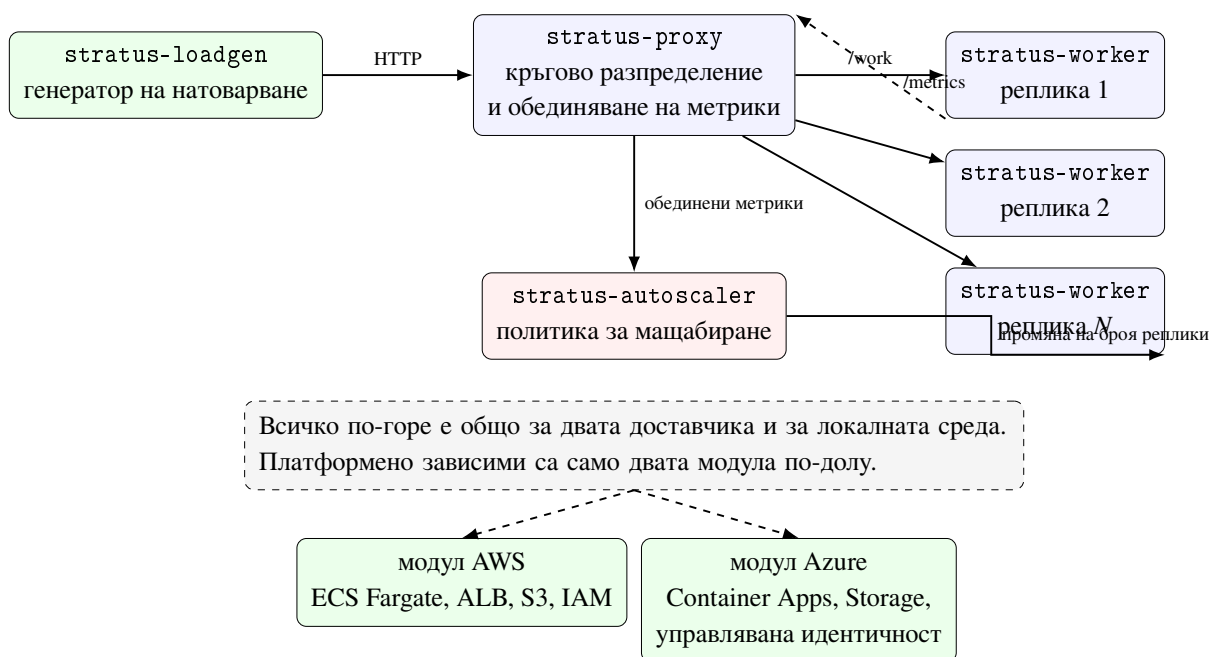
3.1. Изпълними файлове и потоци от данни

Реализацията се състои от пет изпълними файла, изградени от една обща библиотека:

- `stratus-worker`, изчислителният работник с три крайни точки;
- `stratus-proxy`, обратният посредник, който разпределя заявките кръгово и същевременно обединява метриките на целия набор от реплики;
- `stratus-loadgen`, генераторът на синтетично натоварване;
- `stratus-autoscaler`, контролерът за мащабиране;

- `stratus-cost`, калкулаторът за цена на единица работа.

Потокът е следният. Генераторът изпраща заявки към посредника. Посредникът избира реплика по кръгов ред и препраща заявката към нея. Всеки работник поддържа собствени броячи и хистограма и ги излага в текстов формат. Посредникът чете метриките на всички реплики и ги сумира в един документ, от който етикетът с името на репликата е премахнат, така че се получава общата картина на целия набор. Контролерът чете този общ документ на равни интервали, изчислява натоварването от разликата между две последователни показания и решава дали да промени броя на репликите. Схемата е дадена на Фиг. 1 (Фиг. 1).



Фигура 1. Структура на системата и границата между общата и платформено зависимата част

3.2. Единица работа

Сравнението по цена изисква знаменател, който не зависи от средата. За такъв е приета *една единица работа*: едно успешно изпълнение на `/work` със зададени параметри. Изчислението е оценка на функцията за време на бягство на множеството на Манделброт върху решетка с размер $n \times n$ точки и таван от m итерации над фиксиран правоъгълник от комплексната равнина. Сложността е $O(n^2m)$ в най-лошия случай, а действително изпълненият брой итерации се връща в отговора, защото част от точките напускат кръга на бягство по-рано.

Прозорецът в комплексната равнина е фиксиран в изходния текст. Това не е дребна подробност: преместването му би променило обема работа при същите параметри и би обезсилило всяко по-ранно измерване, без нищо във вход или изход да покаже промяната.

За стандартна единица е приета двойката $n = 384$, $m = 500$. При нея се изпълняват точно 18 690 064 итерации от 73 728 000 възможни, като стойността е детерминистична и се проверява от модулен тест.

3.3. Метрика, която управлява мащабирането

Използваната метрика е делът заето време спрямо наличния капацитет. Работникът излага монотонен брояч `stratus_busy_seconds_total`, който натрупва времето, прекарано вътре в изчислителното ядро, и измерител `stratus_worker_threads` с размера на пула. Контролерът изчислява

$$\text{натоварване} = \frac{\Delta \text{busy_seconds}}{\Delta t \cdot \sum \text{worker_threads}}$$

тоест по същия начин, по който всяка система за наблюдение получава скорост от брояч. Работникът нарочно не изчислява това отношение сам: ако го правеше, отношението щеше да носи неговия прозорец на осредняване, а не прозореца на контролера, и двата щяха да се разминават при всяка промяна на интервала на управление.

Натоварването на процесора не се използва като сигнал. Задачата насища ядро по построение, така че натоварването на ядро остава близо до единица и когато наборът от реплики се справя, и когато е задръстен.

3.4. Политика за мащабиране

Политиката е следене на цел с две допълнения. Желаният брой реплики е

$$r_{\text{жел}} = \left\lceil r_{\text{тек}} \cdot \frac{\text{натоварване}}{\text{целево натоварване}} \right\rceil,$$

ограничен до интервала от минималния до максималния допустим брой. Първото допълнение е *зона на нечувствителност*: желание, което се различава от текущия брой с по-малко от зададен дял, се приема за шум и не води до действие. Без нея контролерът трепти с една реплика без край. Второто допълнение са *несиметрични периоди на изчакване*: увеличаването е позволено след кратко изчакване, намаляването

само след по-дълго. Причината е в цената на грешката. Закъсняла реакция при нарастващо натоварване се плаща от всяка заявка в опашката, докато закъсняло освобождаване се плаща с една реплика.

3.5. Договор на модулите за инфраструктура

Двата модула приемат един и същ обект, който описва услугата: име, контейнерен образ, порт, изчислителен ресурс, памет, минимален и максимален брой реплики, целево натоварване, променливи на средата и етикети. Двамата модула връщат едни и същи изходи: адрес на входната точка, име на обектното хранилище, идентификатор на изчислителната платформа и име на идентичността, с която работникът достига хранилището. Заради този еднакъв договор преминаването от единия доставчик към другия е смяна на стойността на една променлива.

3.6. Съпоставяне на изчислителния ресурс

Пряко еднакви типове възли при двамата доставчици не съществуват, затова съпоставянето е записано преди измерванията, а не след тях. Изчислителният ресурс се задава в единиците на AWS Fargate, където 1024 единици са едно виртуално ядро. Модулът за Azure дели на 1024 и получава дробния брой ядра, който Container Apps очаква. Паметта се задава в мегабайти и модулът за Azure я превръща в гигабайти. Избрана е единица с дефинирано преобразуване в двете посоки, за да не се налага в описанието на услугата да се появи стойност, вярна само при единия доставчик.

3.7. Модел на идентичността и достъпа

Работникът никога не притежава дълготраен ключ. При AWS това е роля на задачата, която контейнерът поема през крайната точка за метаданни: маркерът е краткотраен, а правата, които носи, са четене, запис и изтриване на обекти в една конкретна кофа и изброяване на съдържанието ѝ. При Azure това е потребителски присвоена управлявана идентичност с присвоена роля за запис на данни в един конкретен профил за съхранение, като ключовете за споделен достъп на профила са изключени, тоест дълготраен секрет изобщо не съществува.

Този модел изравнява двете разгръщания по нещо съществено: и при двамата

доставчици в изходния текст на проекта не съществува секрет.

Важна уговорка, част първа. Изчислителното натоварване е чисто изчислително и не чете и не записва обекти. Двата модула създават хранилището и подават името му и данните за идентичността на контейнера чрез променливи на средата, но в самия работник няма клиент за обектно хранилище. Причината е, че всяко обръщение към хранилище щеше да внесе мрежово чакане в единица работа, чиято цел е да бъде изцяло изчислителна и с известна сложност. Обвързването на идентичността остава част от описанието на инфраструктурата и е готово за приложение, но не е упражнявано от кода.

Важна уговорка, част втора. Описаният в този параграф модел на достъпа е *написан*, но не е *прилаган*. Няма достъп до сметка при нито един от двамата доставчици и terraform не е инсталиран на машината, върху която е изпълнена работата. Всичко, което може да се твърди за двата модула, е че са написани спрямо документираните схеми на ресурсите. Вж. Раздел V за обхвата на действително проведените измервания.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Изходният текст е разделен по отговорност, а не по вид на файловете. Директорията `include/stratus/` съдържа заглавните файлове с логиката, `src/` реализациите и главните функции, `tests/` модулните тестове, `infra/` описанието на инфраструктурата, а `docs/` настоящия документ. Общо това са 19 файла на C++ с 2724 реда.

Проектът се изгражда с нула задължителни външни зависимости. Достатъчни са компилатор на C++20 и CMake, без изтегляне на пакети при конфигуриране. Това е нарочно решение, а не удобство: хранилището е публично, и изграждане, което изисква мениджър на пакети, е изграждане, което никой не изпълнява. Двете места, където обичайно се появява зависимост, са рамката за тестове и рамката за измерване на време. И двете са заменени с малки собствени реализации, описани по-долу.

4.1. Изчислителното ядро

Ядрото е в `include/stratus/mandelbrot.hpp` и е свободно от заделяне на памет. Функцията `escape_time` връща броя итерации до напускане на кръга с радиус две, ограничен отгоре, а `compute` я прилага върху решетката и връща две числа: контролна сума и точния брой изпълнени вътрешни итерации. Контролната сума има двойна роля: позволява отговорът да бъде проверен и не позволява на оптимизатора да премахне цикъла, чийто резултат иначе никой не чете.

4.2. Работникът

Работникът е в `src/worker_main.cpp` и излага три крайни точки:

- GET `/healthz`, проверка за живост;
- GET `/work?size=&iter=`, една единица работа, с проверени граници на двата параметъра;
- GET `/metrics`, метриките в текстовия формат за представяне на Prometheus.

Параметър извън допустимия обхват води до отговор с код 400, а не до подмяна с най-близката допустима стойност. Тихата подмяна би означавала, че измерване, направено с отхвърлен параметър, изглежда успешно и е сравнявано с други при друг обем работа.

HTTP сървърът е в `src/httpd.cpp` и използва пул с фиксиран размер, а не нишка на

връзка. Изборът е съществен за целия експеримент: при нишка на връзка действителната едновременност е тази, която клиентът е избрал, процесорът се презаписва, а сигналът за натоварване, който управлява контролера, престава да означава каквото и да било. При фиксиран пул капацитетът на една реплика е точно броят нишки в пула.

4.3. Преносимост на входно изходния слой

Единствената платформено зависима част на приложението е `src/net.cpp`. Winsock и интерфейсът за сокети на BSD се различават по инициализация, по типа на дескриптора, по функцията за затваряне и по начина на съобщаване на грешки. Тези четири разлики са поети на едно място, зад заглавния файл `include/stratus/net.hpp`. Всичко над него, тоест работникът, посредникът, генераторът и контролерът, е преносим C++20 без нито един условен блок за платформа.

4.4. Обратният посредник и обединяването на метрики

Посредникът, `src/proxy_main.cpp`, изпълнява две задачи, защото и на двете им трябва едно и също нещо: текущият списък с живи реплики. Списъкът се получава чрез разрешаване на името на услугата: в мрежа на контейнери вграденият DNS връща по един адрес за всяка работеща реплика. Няма регистър на услугите и няма презареждане на конфигурация. Промяната на броя реплики променя това, което отговаря DNS, и посредникът я следва.

Разпределението е кръгово, чрез един общ брояч. При обединяването на метриките посредникът чете документите на всички реплики, премахва етикета с името на репликата и сумира сериите с еднакъв ключ. Премахването на етикета е точно операцията, която превръща N отделни серии в една обща за целия набор, а контролерът управлява именно целия набор.

Редовете `# HELP` и `# TYPE` се извеждат по веднъж за име на метрика, не по веднъж за серия. Метриката `stratus_requests_total` има четири различни набора етикета, и без това правило документът съдържа четири повтарящи се заглавия, което го прави невалиден за всяка система за наблюдение.

4.5. Контролерът за мащабиране

Контролерът, `src/autoscaler_main.cpp`, е нарочно тънък. Самото решение е чиста функция в `include/stratus/scaling.hpp` и се проверява от пет модулни теста без работещ стек от контейнери. Контролен цикъл, който може да бъде наблюдаван само чрез гледане на живо разгърнат клъстер, не може нито да бъде обсъждан, нито проверяван.

Едно място в цикъла заслужава коментар, защото първата му версия беше сгрешена. Общата метрика на набора е сумата от броячите на всички реплики, така че премахването на реплика кара тази сума да *спадне*. Първата версия прочете отрицателната разлика като отрицателно натоварване и реши, съвсем правилно спрямо входа си и съвсем безполезно спрямо действителността, че трябва да намали броя реплики още повече. Правилното поведение е това на всяка система за наблюдение: отрицателна разлика в брояч е нулиране, интервалът не носи използвана скорост, отправната точка се премества напред и интервалът се пропуска.

4.6. Генераторът на натоварване

Генераторът, `src/loadgen_main.cpp`, има два режима, защото отговарят на различни въпроси. При `--rate 0` контурът е затворен: всяка нишка изпраща следващата заявка веднага след като предишната се върне, тоест измерва се колко работа услугата може да поеме. При зададена скорост контурът е отворен: моментите на изпращане са определени предварително, така че услуга, която се забавя, натрупва време на чакане в отчетеното закъснение вместо тихо да ограничава клиента. Вторият режим е този, който показва за какво служи контролерът.

Персентилите се изчисляват върху пълната извадка, а не от кофите на хистограма. Извадката се побира в паметта при използваните скорости, а отчитането на границата на кофа като измерен персентил е точно онова тихо закръгляне, което кара две различни системи да изглеждат еднакви.

4.7. Моделът на разхода

Калкулаторът, `include/stratus/cost.hpp` и `src/cost_main.cpp`, приема всяка цена като задължителен вход и отказва да работи без нея. В изходния текст няма нито една публикувана цена и не може да бъде добавена: цените се различават по регион, по

ангажимент, по сметка и по месец, така че цена, вписана в програма, е число, което е невярно още на следващия ден. Компонент, за който е дадена цена, но не и количество, се отчита като изключен, а не като нулев.

4.8. Модулни тестове

Рамката за тестове е един заглавен файл, `tests/test_framework.hpp`, под сто и петдесет реда: регистър, твърдение, което отпечатва изрази и мястото, и възможност да се изпълни един тест по име. Последното позволява всеки тест да бъде отделен запис в `CTest`, така че при провал се съобщава кой тест е паднал, а не само че двоичният файл е върнал ненулев код.

Тестовите са 26 и покриват детерминизма и границата на цената на изчислителното ядро, разбора на заявките и на параметрите, представянето и обратното разчитане на метриките, обединяването между реплики, петте случая на политиката за мащабиране, модела на разхода и статистиката за закъснението. Гетъри не се тестват.

4.9. Контейнеризация

Контейнерният образ се изгражда на два етапа. Първият носи компилатор, `CMake` и `make` и свързва изпълнимите файлове статично срещу `musl`. Вторият е `scratch` и съдържа само петте изпълними файла. Няма мениджър на пакети, няма обвивка и няма библиотека, която да се обновява. Цената е, че в работещ контейнер няма с какво да се човърка, което е приемлива размяна за приложение, чийто единствен интерфейс са три крайни точки по `HTTP`. Измереният размер на получения образ е **8,09 MB**.

Модулните тестове се изпълняват вътре в етапа на изграждане. Образ, който се е компилирал, но не минава собствените си тестове, не бива да съществува.

Един детайл струваше един неуспешен опит. Файлът за изграждане задава `CMD`, а не `ENTRYPOINT`. Един образ носи петте изпълними файла и файлът за композиране избира по един за всяка услуга чрез `command`. При `ENTRYPOINT` този избор се превръща в аргументи, подадени на работника, и трите услуги се стартират като работници, без нищо да съобщи за грешка.

4.10. Описание на инфраструктурата

Кореновата конфигурация избира модул според стойността на променлива за доставчик и подава към него един и същ обект. Изразите с `count` в `infra/main.tf` са единственото място в цялата конфигурация, където доставчик се назовава извън собствения му модул.

Количествената мярка за преносимост на описанието е следната, преброена в редове без празни и без коментари: общата част е 82 реда, модулът за AWS е 297 реда, модулът за Azure е 178 реда. Съотношението е показателно и не е в полза на твърдението, че едно описание покрива двата доставчика. Общото описание е кратко, защото описва *услугата*, а всичко останало, тоест мрежа, платформа, хранилище и идентичност, остава платформено зависимо и се пише два пъти.

Разликата между двата модула по обем идва почти изцяло от мрежата. При Azure Container Apps мрежата и балансировчикът са част от средата, докато при AWS Fargate се създават изрично виртуална мрежа, подмрежи, маршрутна таблица, интернет шлюз, две групи за сигурност и балансировчик с целева група.

Състояние. Двата модула са написани и форматирани, но **не са прилагани**. На машината няма `terraform` и няма данни за достъп до нито една сметка. Твърдението за тях се изчерпва с това.

4.11. Мерки срещу изтичане на данни за достъп

В хранилището не се записват идентификатори на сметки, абонаменти, имена на ресурси с глобален обхват или ключове. Всички такива стойности се подават чрез променливи на средата, а файлът `.env.example` съдържа само имената им със заместващи стойности. Файловете със състояние и с локални променливи са изключени от версионизирането.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНО СРАВНЕНИЕ

Сравнението между двама доставчици има смисъл само ако разликата в измереното може да бъде отдадена на средата, а не на разликите в настройката. Затова методиката се записва преди провеждането на измерванията и всяко отклонение от нея се отбелязва явно в Раздел VI.

5.1. Обхват на действително проведените измервания

Този параграф стои преди останалите, защото определя какво изобщо може да се твърди.

Не е разгръщано в облак. На машината, върху която е изпълнена работата, не е инсталиран terraform и няма данни за достъп до сметка нито при AWS, нито при Azure. Двата модула за инфраструктура са написани и форматирани, но не са прилагани. Следователно *нито едно* число за закъснение, за поведение при мащабиране или за цена при кой да е от двамата доставчици не съществува и не се съобщава. Всяко такова място в Раздел VI е оставено с маркер TODO вместо да бъде запълнено с правдоподобна стойност.

Измерено е локално. Проведени са три експеримента върху локален стек от контейнери на една машина. Те не отговарят на въпроса за разликата между доставчиците. Те отговарят на друг въпрос, който също е част от целта: работи ли построената система, как се държи капацитетът ѝ при промяна на броя реплики, и реагира ли контролерът за мащабиране на действителна метрика в разумно време.

5.2. Конфигурация на машината

Всички локални резултати са получени при следната конфигурация, записана тук, за да може измерването да бъде повторено:

Таблица 2. Конфигурация на средата за измерване

Елемент	Стойност
Операционна система	Windows 11 Pro N, версия 10.0.26200
Логически процесори	12
Компилатор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Стандарт	C++20, режим Release
Система за изграждане	CMake 4.3.2 с Ninja 1.13.2
Контейнерна среда	Docker 29.4.3, Docker Compose v5.1.3
Основа на образа	Alpine Linux 3.20 при изграждане, scratch при изпълнение
Свързване в образа	статично, срещу musl

Съществено обстоятелство: генераторът на натоварване се изпълнява на *същата* машина, върху която работят контейнерите, и дели с тях същите 12 логически процесора. Това е основният източник на разсейване в локалните резултати и е отбелязано при тълкуването им.

5.3. Параметри на натоварването

Единицата работа е фиксирана за всички измервания: $n = 384$ точки на страна и таван $m = 500$ итерации, тоест 18 690 064 действително изпълнени вътрешни итерации. Всяка реплика има ограничение от едно виртуално ядро и една нишка за обработка на заявки, така че капацитетът на набора е точно равен на броя реплики.

Всяко измерване има период на загряване от 5 секунди, чиито резултати се отхвърлят, и период на измерване от 20 секунди. Всяка точка се повтаря 3 пъти, а всяко повторение се записва отделно. Нищо не се осреднява преди да достигне до файл: в Раздел VI се съобщава медианата, но и трите стойности са налични.

5.4. Условия за валидност на едно измерване

Едно локално измерване се приема за валидно, ако са изпълнени всички следни условия едновременно: всички реплики използват контейнерен образ с една и съща контролна сума; броят реплики е потвърден чрез крайната точка /backends на посредника преди началото на измерването; генераторът е един и същ и се изпълнява от една и съща машина; единицата работа е неизменена; и измерването започва след изтичане на периода на загряване. Измерване, което не отговаря на някое от тези условия, не се сравнява.

За бъдещо сравнение между двама доставчици към горните се добавят: съпоставените изчислителни ресурси са тези, определени в Раздел III; регионите са географски съпоставими; и правилото за мащабиране има еднакви прагове и еднакви граници.

5.5. Проведени експерименти

Експеримент 1, капацитет спрямо брой реплики. Броят реплики се задава на 1, 2, 4 и 6. При всеки от тях се пуска натоварване в затворен контур с 2 едновременни заявки на реплика. Затвореният контур е избран нарочно: при него отчетената пропускателна способност е мярка за капацитет, а не мярка за търпението на клиента. Отчитат се пропускателна способност и закъснение по персентили. Команда:

```
powershell -File bench.ps1
```

Експеримент 2, време за реакция при увеличаване. Наборът се увеличава от 1 на 2 реплики и се измерва времето от подаването на командата до момента, в който посредникът започва да насочва заявки към новата реплика. Повторенията са 5. Измереното време включва и закъснението от откриването на репликата през DNS, което при използваната настройка е до 1 секунда. Тази съставка не е извадена, а е посочена.

Експеримент 3, поведение на контролера под натоварване. Стекът се вдига с една реплика, стартира се контролерът, и срещу набора се пуска натоварване в отворен контур със скорост, която една реплика не може да поеме. Записва се времевата редица от решения на контролера заедно с метриката, която е предизвикала всяко от тях. Команда:

```
powershell -File demo.ps1
```

Експеримент 4, цена за единица работа. *Не е провеждан.* Той изисква начислена от доставчик сума, каквато не съществува. Реализиран е инструментът, който изчислява цената при подадени цени и измерена пропускателна способност, но самата сметка не може да бъде получена без разгръщане.

5.6. Отчитани величини

За всеки експеримент се отчитат: закъснение по персентили, а не средна стойност, защото при опашки средната стойност крие точно това, което ни интересува;

пропускателна способност в изпълнени единици работа за секунда; брой активни реплики във времето; и дял на неуспешните заявки. Всяка величина се записва в отделен файл в `results/` заедно с параметрите, при които е получена.

5.7. Известни ограничения на методиката

Локалните измервания са направени на машина, която едновременно изпълнява генератора, посредника и всички реплики. При 6 реплики броят готови за изпълнение нишки надвишава броя логически процесори, тоест системата е презаписана и част от измереното закъснение идва от планировчика на операционната система, а не от услугата. Това е ограничение на средата, не на реализацията, и е причината всяка точка да се повтаря по три пъти.

Docker Desktop под Windows изпълнява контейнерите във виртуална машина. Между генератора и работника стои допълнителен мрежов слой, който добавя закъснение с непостоянна стойност.

Методиката за сравнение между доставчици не изолира мрежовото разстояние между генератора и двата региона, което влияе на абсолютните стойности на закъснението. Тя не изолира и евентуалното различие в поколението на процесора между съпоставените изчислителни ресурси, което е извън контрола на потребителя при двамата доставчици.

[TODO: Да се прецени дали мрежовото разстояние да бъде извадено от отчетеното закъснение чрез отделно измерване на времето за обръщение до всяка входна точка, и ако да, да се опише как. Въпросът става актуален едва при действително разгръщане.]

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя получените резултати в реда на експериментите от Раздел V. Всяко число тук е измерено на машината, описана в Табл. 2, с командите, посочени в Раздел V, и се намира в суровите файлове в `results/`. Числа, които не са измерени, не са попълнени.

6.1. Капацитет спрямо брой реплики

Табл. 3 съдържа и трите повторения на всяка точка, а не само обобщение (Табл. 3). Разсейването между повторенията е част от резултата и скриването му би направило таблицата по-подредена и по-невярна.

Таблица 3. Пропускателна способност и закъснение по персентили спрямо броя реплики. Затворен контур, 2 едновременни заявки на реплика, 5 s заграване, 20 s измерване

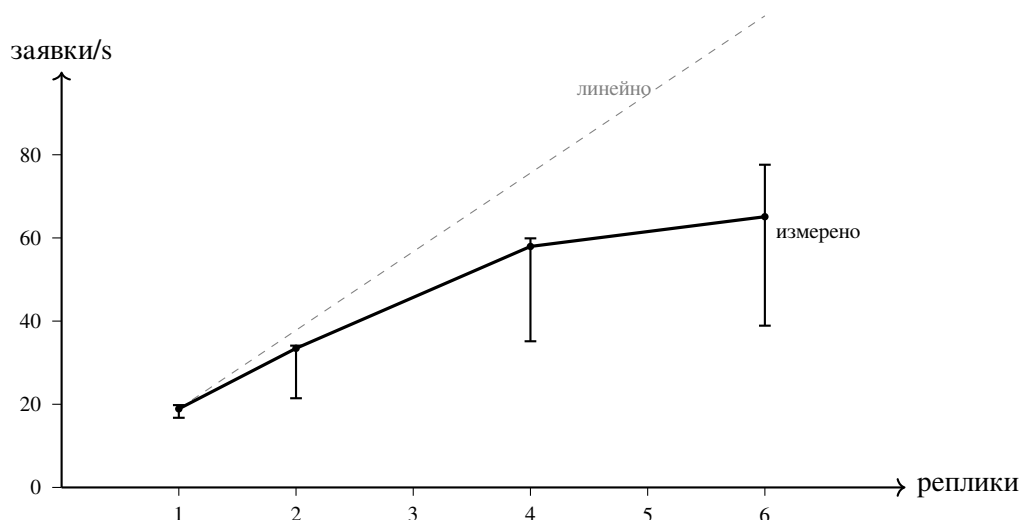
Реплики	Повт.	Едновр.	Пропуск. [зая./s]	p50 [ms]	p90 [ms]	p95 [ms]	p99 [ms]
1	1	2	16,75	99,01	169,25	255,31	411,99
1	2	2	19,80	99,25	106,20	109,44	120,55
1	3	2	18,90	100,15	115,88	128,00	203,35
2	1	4	33,45	117,52	175,72	207,39	285,80
2	2	4	21,45	179,52	282,37	321,73	433,96
2	3	4	34,10	129,18	183,66	202,74	244,84
4	1	8	35,15	161,06	493,66	611,11	711,22
4	2	8	59,90	104,38	232,10	304,19	462,00
4	3	8	57,95	118,35	222,94	299,56	399,33
6	1	12	65,10	115,53	390,38	523,00	859,78
6	2	12	77,60	120,22	244,28	353,46	888,83
6	3	12	38,90	234,45	617,65	743,56	863,21

Неуспешни заявки няма: и в дванадесетте повторения делът им е нула.

Обобщението по медиана е дадено в Табл. 4 (Табл. 4).

Таблица 4. Медиана от трите повторения и ускорение спрямо една реплика

Реплики	Пропуск. [зая./s]	Ускорение	p50 [ms]	p90 [ms]	p99 [ms]
1	18,90	1,00	99,25	115,88	203,35
2	33,45	1,77	129,18	183,66	285,80
4	57,95	3,07	118,35	232,10	462,00
6	65,10	3,44	120,22	390,38	863,21



Фигура 2. Пропускателна способност спрямо броя реплики. Точките са медианата от три повторения, отсечките показват най-малката и най-голямата стойност

Коментар. Мащабирането е подлинейно и се изчерпва (Фиг. 2). От 1 до 4 реплики ускорението е 3,07 при четирикратно увеличение на ресурса, тоест ефективността е около 77%. От 4 до 6 реплики пропускателната способност нараства само с 12%, а закъснението на 99-и персентил се удвоява, от 462 ms на 863 ms. Обяснението не е в услугата: при 6 реплики върху машина с 12 логически процесора едновременно работят 6 изчислителни нишки, 64 нишки на посредника и 12 нишки на генератора, и системата е презаписана. Измерва се планировчикът на операционната система, не капацитетът на услугата.

Разсейването нараства заедно с броя реплики. При една реплика разликата между най-ниската и най-високата стойност е 3,05 заявки/s, при шест реплики е 38,70 заявки/s, тоест почти колкото цялата медиана при две реплики. Този резултат е и основната причина локалните числа да не се представят като характеристика на услугата: те са характеристика на конкретната машина под конкретно натоварване.

Закъснението на 50-и персентил остава почти постоянно, между 99 и 130 ms, докато закъснението на високите персентили расте многократно. Това е типичното поведение на

система с опашки и е точно причината в Раздел V да е записано, че се отчитат персентили, а не средна стойност: средното закъснение при 6 реплики би скрило 865 ms опашка зад 120 ms медиана.

6.2. Време за реакция при увеличаване на набора

Таблица 5. Време от подаване на командата за увеличаване до насочване на трафик към новата реплика, 5 повторения

Повторение	От	До	Време [s]
1	1	2	4,662
2	1	2	5,234
3	1	2	1,357
4	1	2	2,001
5	1	2	1,840
медиана			2,001
най-малка, най-голяма			1,357 ... 5,234

Коментар. Измереното време съдържа три съставки: създаване и стартиране на контейнера, регистрация на новия адрес във вградения DNS, и откриването му от посредника, чийто период на опресняване е 1 секунда. Разделянето им не е направено, затова се съобщава общата стойност. Разликата от близо четири пъти между най-бързото и най-бавното повторение при иначе еднакви условия показва, че времето за създаване на контейнер не е постоянна величина, и че всяка политика за мащабиране, чийто период на управление е от порядъка на секунда, работи с шум от същия порядък. Именно затова периодът на управление в Експеримент 3 е 3 секунди, а не по-малък.

6.3. Поведение на контролера под натоварване

Стекът е вдигнат с една реплика, контролерът е стартиран с цел 0,70, граници от 1 до 6 реплики, период 3 s и периоди на изчакване 6 s при увеличаване и 15 s при намаляване. Натоварването е в отворен контур, 40 заявки в секунда, 32 едновременни, 45 s измерване.

Отчетът на генератора за този период: 1868 успешни заявки, 0 неуспешни, пропускателна способност 41,51 заявки/s, закъснение $p_{50} = 51,84$ ms, $p_{90} = 1508,20$ ms, $p_{95} = 1534,61$ ms, $p_{99} = 1639,47$ ms, най-голямо 1658,75 ms. Разликата между медианата и високите персентили е близо тридесеткратна и не е дефект: тя е опашката, натрупана в

първите секунди, когато една реплика приема натоварване, което не може да поеме. Това е желаният вид резултат в отворен контур. Закъснението показва натрупаното чакане, вместо клиентът тихо да намали скоростта си, а именно натрупаното чакане е онова, което контролерът съществува да съкрати.

Табл. 6 възпроизвежда времевата редица от решения (Табл. 6).

Таблица 6. Решения на контролера и метриката, която е предизвикала всяко от тях

t [s]	Реплики	Натоварване	Желани	Решение
3,0	1	0,000	1	задържане, на целта
6,9	1	0,000	1	задържане, на целта
10,3	1	1,101	2	увеличаване , над целта
15,6	2	0,848	2	задържане, изчакване преди увеличаване
20,2	2	0,822	3	увеличаване , над целта
25,6	3	0,996	3	задържане, изчакване преди увеличаване
30,2	3	0,531	3	задържане, на целта
33,3	3	0,667	3	задържане, на целта
36,3	3	0,654	3	задържане, на целта
39,4	3	0,653	3	задържане, на целта
42,5	3	0,661	3	задържане, на целта
45,6	3	0,660	3	задържане, на целта
48,7	3	0,654	3	задържане, на целта
51,7	3	0,660	3	задържане, на целта
54,8	3	0,655	3	задържане, на целта
57,9	3	0,646	3	задържане, на целта
60,9	3	0,033	1	намаляване , под целта
65,9	1	нулиран брояч		пропуснат интервал
68,9	1	0,000	1	задържане, на целта
... следват още девет интервала на задържане до $t = 99,8$				

Коментар. Контурът се затваря. Натоварването расте, контролерът увеличава набора на две стъпки, от 1 на 2 и от 2 на 3 реплики, след което натоварването се установява между 0,646 и 0,667, тоест непосредствено под целта 0,70, и контролерът спира да действа за десет последователни интервала. След премахването на натоварването той връща набора на една реплика.

Установената стойност позволява независима проверка на цялата верига. При 3 реплики, натоварване 0,655 и измерени 40,0 заявки в секунда, времето за обслужване на една заявка следва да е $0,655 \cdot 3/40,0 = 49,1$ ms. Това съвпада с времето, което

изчислителното ядро отчита само за себе си при същите параметри. Тоест метриката, която управлява контролера, измерва действително извършената работа, а не сумата на чакането по пътя до нея.

Три подробности заслужават внимание, защото са точно нещата, които политиката е предвидена да прави.

Първо, при $t = 15,6$ натоварването 0,848 е над целта, но периодът на изчакване след предишната промяна не е изтекъл и решението е задържане. Същото се случва при $t = 25,6$ със стойност 0,996. Без периодите на изчакване контролерът щеше да добави реплики на всеки интервал, докато първите още се стартират, и да надхвърли многократно нужния брой, преди изобщо да види резултата от първото си действие.

Второ, при $t = 60,9$ натоварването пада на 0,033 и решението е намаляване направо до долната граница. Скокът от 3 на 1 реплика не е грешка: желаният брой при това натоварване е под единица, а долната граница е 1.

Трето, редът при $t = 65,9$ показва пропуснат интервал заради нулиран брояч. Общата метрика на набора е сумата от броячите на репликите, така че при премахване на реплика тя спада. Първата версия на контролера четеше отрицателната разлика като отрицателно натоварване и намаляваше набора още повече. Обработката на нулирането е описана в Раздел IV.

Зоната на нечувствителност не се задейства в това конкретно изпълнение, тъй като установеното натоварване попада точно в интервала, при който желаният брой съвпада с текущия. Поведението ѝ се проверява от модулен тест, а не от това измерване, и точно затова политиката е отделена в чиста функция.

6.4. Размер на контейнерния образ

Измереният размер на получения образ е 8,09 MB, отчетен с `docker image ls`. Той съдържа пет статично свързани изпълними файла и нищо друго: няма мениджър на пакети, няма обвивка и няма споделена библиотека.

6.5. Цена за единица извършена работа

[TODO: Таблица със структура: доставчик, компонент на разхода, начислена сума за периода, брой изпълнени единици работа, цена за единица. Не е попълнена, защото не съществува начислена от доставчик сума: нищо не е разгръщано.]

Инструментът `stratus-cost` изчислява таблицата при подадени цени и измерена пропускателна способност, но в него няма вградена цена и такава няма да бъде добавена.]

[TODO: Коментар: коя среда излиза по-евтина за същата работа, кой компонент отговаря за разликата, и променя ли се изводът при по-голям обем. Изисква Експеримент 4.]

6.6. Сравнение между двамата доставчици

[TODO: Таблица със структура: доставчик, брой заявки в секунда, закъснение на 50-и, 90-и, 95-и и 99-и персентил, дял на неуспешните заявки. Не е попълнена: описанието на инфраструктурата не е прилагано, вж. Раздел 5.1.]

[TODO: Таблица със структура: доставчик, време от преминаването на прага до създаване на нова реплика, време до готовност, време до поемане на трафик, общо време за реакция. Не е попълнена по същата причина.]

6.7. Обобщен анализ

Върху въпроса, поставен в увода, локалните резултати позволяват да се каже следното.

Построената система работи и контурът се затваря: метрика, изнесена от приложението, управлява броя реплики, и решенията са проследими до числото, което ги е предизвикало (Табл. 6). Политиката е чиста функция и се проверява без работещ клъстер, което е и причината трите ѝ поведения, тоест зоната на нечувствителност, несиметричните периоди на изчакване и обработката на нулиран брояч, да могат да бъдат обсъждани по измерени редове, а не по общи съображения.

Мащабирането в използваната среда е подлинейно и се изчерпва при 4 реплики, но причината е известна и е в средата, не в услугата: 12 логически процесора, върху които работят и генераторът, и посредникът, и репликите.

За твърдението „едно декларативно описание е достатъчно за двама доставчици“ проектът дава частичен и отрицателен отговор, при това без да е разгръщал каквото и да било. Общото описание на услугата е 82 реда, а двата платформено зависими модула са 297 и 178 реда. Тоест дели се *описанието на услугата*, а не описанието на средата. Това е резултат от преброяване на редовете в написаното описание, не от измерване при

работещо разгръщане, и е единственото, което може да се твърди преди прилагане.

[TODO: След действително разгръщане: доколко от разликата между двамата доставчици е видима за потребителя на услугата, и колко от нея се вижда още в описанието на инфраструктурата.]

ЗАКЛЮЧЕНИЕ

Проектът изгражда изчислително интензивна услуга, средствата за нейното наблюдение и мащабиране, и описание на инфраструктурата, което насочва една и съща услуга към двама публични облачни доставчика чрез стойността на една променлива. Разделението между общата и платформено зависимата част е направено така, че различията между доставчиците да са локализирани в два модула с еднакъв интерфейс.

Какво е проверено и какво не. Изградената система е измерена локално: капацитет спрямо брой реплики, време за реакция при увеличаване и поведение на контролера за мащабиране под натоварване, всяко с повторения и със записани сурови данни. Описанието на инфраструктурата **не е прилагано**, защото няма данни за достъп до сметка при нито един от двамата доставчици и terraform не е инсталиран на използваната машина. Следователно измерване на разликата *между* двете среди в този документ няма, и такова не е съчинено. Съответните места в Раздел VI стоят празни.

Предимства на избраното решение. Метриките се събират от самото приложение, с един и същ код при всяко разгръщане, така че разликата в измереното не се дължи на разлика в измервателния уред. Политиката за мащабиране е чиста функция и се проверява без работещ клъстер, което позволи трите ѝ нетривиални поведения да бъдат обсъдени по измерени редове. Проектът се изгражда без нито една задължителна външна зависимост, а полученият контейнерен образ е 8,09 MB и не съдържа нищо освен изпълнимите файлове. В хранилището с изходния текст не се съхранява нито един секрет.

Недостатъци и ограничения. Твърдението „едно описание за двама доставчици“ се оказва по-слабо, отколкото звучи. Преброяването показва 82 реда общо описание срещу 297 и 178 реда платформено зависими модула, тоест дели се описанието на *услугата*, а не на средата. Съпоставянето на изчислителния ресурс между двамата доставчици е приблизително, защото еднакви типове не съществуват, и е фиксирано предварително в Раздел III. Локалните измервания са направени на машина, която едновременно изпълнява генератора, посредника и всички реплики, тоест при 6 реплики системата е презаписана и част от измереното закъснение идва от планировчика на операционната система. Мрежовото разстояние до двата региона не е изолирано, но този въпрос става актуален едва при действително разгръщане.

Едно очакване, което не се потвърди. Първоначалният избор беше управляван

Kubernetes, с обосновката, че описанието на работното натоварване съвпада буквално при двамата доставчици. При реализацията се оказва, че построената услуга не използва нищо от Kubernetes освен способността му да държи N реплики достъпни на едно име, а добавя втори слой на мащабиране, който замъглява точно величината, която проектът измерва. Изборът беше преразгледан в полза на управлявани услуги за контейнери, а обосновката за това е записана в Раздел II заедно с първоначалната, вместо първоначалната да бъде премълчана.

Насоки за по-нататъшна работа. Първата и очевидна стъпка е действително прилагане на двата модула и попълване на празните места в Раздел VI. След това: разширяване на сравнението с трети доставчик, добавяне на второ, входно изходно интензивно натоварване, при което гаранциите на обектното хранилище влизат в сила по различен начин, както и проверка на поведението при отказ на цяла зона на достъпност.

ЛІТЕРАТУРА

- [1] P. Mell and T. Grance, “The NIST definition of cloud computing,” Tech. Rep. Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD, USA, September 2011.
- [2] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [3] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, “Xen and the art of virtualization,” in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pp. 164–177, 2003.
- [4] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.
- [5] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, pp. 419–434, 2020.
- [6] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proceedings of the 10th European Conference on Computer Systems (EuroSys '15)*, 2015.
- [7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” *ACM Queue*, vol. 14, no. 1, 2016.
- [8] I. Foster, C. Kesselman, and S. Tuecke, “The anatomy of the grid: Enabling scalable virtual organizations,” *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07)*, pp. 205–220, 2007.

- [10] B. Calder, J. Wang, A. Ogus, N. Nilakantan, A. Skjolsvold, S. McKelvie, Y. Xu, S. Srivastav, J. Wu, H. Simitci, *et al.*, “Windows Azure storage: A highly available cloud storage service with strong consistency,” in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11)*, pp. 143–157, 2011.
- [11] HashiCorp, “Terraform documentation.” <https://developer.hashicorp.com/terraform/docs>.
посетен на [TODO: дата на достъп].
- [12] The Kubernetes Authors, “Kubernetes documentation.” <https://kubernetes.io/docs/>.
посетен на [TODO: дата на достъп].

ПРИЛОЖЕНИЯ

Приложените листинги са съществени части от изходния текст. Пълният текст е в хранилището на проекта.

Приложение А. Изчислителното ядро

Единицата работа. Прозорецът в комплексната равнина е фиксиран нарочно, защото преместването му би променило обема работа при същите параметри.

```
1 // The unit of work. A deterministic, CPU bound, allocation free kernel whose
   ↪ cost is
2 // known in advance from its parameters, which is what makes it usable as the
   ↪ denominator
3 // of a cost per unit of work figure.
4 //
5 // The kernel samples the Mandelbrot escape time function on a size x size
   ↪ grid over a
6 // fixed window of the complex plane, with a fixed iteration ceiling. It
   ↪ returns both a
7 // checksum (so a response can be verified and so the compiler cannot elide
   ↪ the loop) and
8 // the exact number of inner iterations performed (so throughput can be
   ↪ reported in real
9 // arithmetic operations rather than in requests, which say nothing without
   ↪ their size).
10 #pragma once
11
12 #include <stdint>
13 #include <cstdint>
14
15 namespace stratus {
16
17 struct WorkResult {
18     std::uint64_t checksum = 0;    // sum of per-pixel escape counts
19     std::uint64_t iterations = 0;  // inner loop iterations actually executed
20 };
21
```

```

22 // Window of the complex plane. Fixed on purpose: moving it would change the
    ↪ amount of
23 // work for the same size and silently invalidate every earlier measurement.
24 inline constexpr double kReMin = -2.0;
25 inline constexpr double kReMax = 0.5;
26 inline constexpr double kImMin = -1.25;
27 inline constexpr double kImMax = 1.25;
28
29 // Escape time for one point, capped at max_iter. Returns the iteration count
    ↪ .
30 constexpr std::uint32_t escape_time(double cr, double ci, std::uint32_t
    ↪ max_iter) noexcept {
31     double zr = 0.0;
32     double zi = 0.0;
33     std::uint32_t n = 0;
34     while (n < max_iter) {
35         const double zr2 = zr * zr;
36         const double zi2 = zi * zi;
37         if (zr2 + zi2 > 4.0) break;
38         zi = 2.0 * zr * zi + ci;
39         zr = zr2 - zi2 + cr;
40         ++n;
41     }
42     return n;
43 }
44
45 // size x size samples, max_iter ceiling. Cost is  $O(\text{size}^2 * \text{max\_iter})$  in the
    ↪ worst case
46 // and strictly bounded by it, which is why iterations is reported rather
    ↪ than assumed.
47 inline WorkResult compute(std::uint32_t size, std::uint32_t max_iter)
    ↪ noexcept {
48     WorkResult out;
49     if (size == 0 || max_iter == 0) return out;
50
51     const double dr = (kReMax - kReMin) / static_cast<double>(size);
52     const double di = (kImMax - kImMin) / static_cast<double>(size);
53
54     for (std::uint32_t y = 0; y < size; ++y) {

```

```

55     const double ci = kImMin + (static_cast<double>(y) + 0.5) * di;
56     for (std::uint32_t x = 0; x < size; ++x) {
57         const double cr = kReMin + (static_cast<double>(x) + 0.5) * dr;
58         const std::uint32_t n = escape_time(cr, ci, max_iter);
59         out.checksum += n;
60         out.iterations += n;
61     }
62 }
63 return out;
64 }
65
66 } // namespace stratus

```

Листинг 1: include/stratus/mandelbrot.hpp

Приложение Б. Политика за мащабиране

Частта, която обикновено е скрита вътре в облачния доставчик. Чиста функция, затова се проверява от модулни тестове без работещ клъстер.

```

1 // The autoscaling policy.
2 //
3 // This is the part a cloud provider normally hides. It is a pure function of
4 // ↪ the observed
5 // utilisation and the elapsed time since the last change, which is exactly
6 // ↪ why it is
7 // worth extracting: a control loop that can only be observed by watching a
8 // ↪ live cluster
9 // cannot be argued about, and cannot be tested.
10 //
11 // The rule is target tracking with a dead band and asymmetric cooldowns:
12 //
13 //     desired = ceil(replicas * utilisation / target)
14 //
15 // clamped to [min, max]. Scaling up is allowed after a short cooldown,
16 // ↪ scaling down only
17 // after a longer one, because the cost of reacting late to a load increase
18 // ↪ is paid by

```



```

14 // every request in the queue, while the cost of releasing a worker late is
    ↪ one worker.
15 #pragma once
16
17 #include <algorithm>
18 #include <cmath>
19 #include <cstdint>
20 #include <string>
21
22 namespace stratus::scaling {
23
24 struct Policy {
25     double target_utilisation = 0.70; // busy fraction of worker capacity we
    ↪ aim for
26     int min_replicas = 1;
27     int max_replicas = 8;
28     double scale_up_cooldown_s = 5.0;
29     double scale_down_cooldown_s = 20.0;
30     // Relative dead band. A desired count within this fraction of the
    ↪ current one is
31     // treated as noise. Without it the loop oscillates by one replica
    ↪ indefinitely.
32     double dead_band = 0.10;
33 };
34
35 enum class Action { Hold, ScaleUp, ScaleDown };
36
37 struct Decision {
38     Action action = Action::Hold;
39     int desired_replicas = 0;
40     double utilisation = 0.0;
41     double raw_desired = 0.0;
42     std::string reason;
43 };
44
45 inline const char* action_name(Action a) {
46     switch (a) {
47         case Action::ScaleUp: return "scale-up";
48         case Action::ScaleDown: return "scale-down";

```

```

49         default: return "hold";
50     }
51 }
52
53 // utilisation is the observed busy fraction of the fleet, in [0, inf).
54 ↪ Values above 1.0
55 // are possible and meaningful: they mean the queue is growing.
56 inline Decision decide(const Policy& p, int current_replicas, double
57 ↪ utilisation,
58                             double seconds_since_last_change) {
59     Decision d;
60     d.utilisation = utilisation;
61     d.desired_replicas = current_replicas;
62
63     const int cur = std::max(1, current_replicas);
64     const double target = (p.target_utilisation > 0.0) ? p.target_utilisation
65     ↪ : 1.0;
66
67     d.raw_desired = static_cast<double>(cur) * utilisation / target;
68     int desired = static_cast<int>(std::ceil(d.raw_desired - 1e-9));
69     desired = std::clamp(desired, p.min_replicas, p.max_replicas);
70
71     if (desired == current_replicas) {
72         d.reason = "at target";
73         return d;
74     }
75
76     // Dead band, expressed on the unclamped desire so that a request to move
77     ↪ from 3.05 to
78     // 3 does not count as a decision.
79     const double relative = std::abs(d.raw_desired - static_cast<double>(cur)
80     ↪ ) /
81
82                             static_cast<double>(cur);
83
84     if (relative < p.dead_band) {
85         d.reason = "within dead band";
86         return d;
87     }
88
89     const bool up = desired > current_replicas;

```

```

83     const double cooldown = up ? p.scale_up_cooldown_s : p.
↪ scale_down_cooldown_s;
84     if (seconds_since_last_change < cooldown) {
85         d.reason = up ? "scale-up blocked by cooldown" : "scale-down blocked
↪ by cooldown";
86         return d;
87     }
88
89     d.action = up ? Action::ScaleUp : Action::ScaleDown;
90     d.desired_replicas = desired;
91     d.reason = up ? "utilisation above target" : "utilisation below target";
92     return d;
93 }
94
95 // Utilisation from two consecutive scrapes of a monotonically increasing
↪ busy time
96 // counter. This is how a scraper derives a rate, and it is the reason the
↪ worker exposes
97 // busy seconds as a counter rather than exposing a ratio it computed itself:
↪ a ratio
98 // computed inside the worker would carry the worker's own window, not the
↪ controller's.
99 inline double utilisation_from_delta(double busy_seconds_delta, double
↪ wall_seconds_delta,
100                                     int worker_threads_total) {
101     if (wall_seconds_delta <= 0.0 || worker_threads_total <= 0) return 0.0;
102     const double capacity = wall_seconds_delta * static_cast<double>(<
↪ worker_threads_total);
103     if (capacity <= 0.0) return 0.0;
104     return busy_seconds_delta / capacity;
105 }
106
107 } // namespace stratus::scaling

```

Листинг 2: include/stratus/scaling.hpp

Приложение В. Модел на разхода

Всяка цена е задължителен вход. В този файл няма нито една публикувана цена.

```
1 // Cost per unit of work.
2 //
3 // Every price is an INPUT. Not one number in this file is a published cloud
  ↳ price, and
4 // none may ever be added: prices differ by region, by commitment, by account
  ↳ and by month,
5 // so a price baked into source is a number that is wrong the day after it is
  ↳ written. The
6 // calculator refuses to run without the prices being supplied explicitly.
7 #pragma once
8
9 #include <optional>
10 #include <string>
11 #include <vector>
12
13 namespace stratus::cost {
14
15 struct Inputs {
16     // Compute
17     double price_per_instance_hour = -1.0; // currency units per instance
  ↳ hour
18     double instances = -1.0; // average instance count over
  ↳ the window
19     // Optional components. Negative means "not supplied", and the component
  ↳ is reported as
20     // excluded rather than silently treated as zero.
21     double price_per_gb_month_storage = -1.0;
22     double storage_gb = -1.0;
23     double price_per_gb_egress = -1.0;
24     double egress_gb = -1.0;
25     // Observed performance
26     double throughput_units_per_second = -1.0; // units of work completed
  ↳ per second
27     double window_seconds = 3600.0;
28 };
```

```

29
30 struct Component {
31     std::string name;
32     double amount = 0.0;
33     bool supplied = false;
34 };
35
36 struct Result {
37     bool ok = false;
38     std::string error;
39     std::vector<Component> components;
40     double total_cost = 0.0;           // cost over the window
41     double units_of_work = 0.0;       // units completed over the window
42     double cost_per_unit = 0.0;
43     double cost_per_million_units = 0.0;
44 };
45
46 inline Result compute(const Inputs& in) {
47     Result r;
48     if (in.price_per_instance_hour < 0.0)
49         return r.error = "price-per-instance-hour not supplied", r;
50     if (in.instances <= 0.0) return r.error = "instances must be greater than
↪ zero", r;
51     if (in.throughput_units_per_second <= 0.0)
52         return r.error = "throughput must be greater than zero", r;
53     if (in.window_seconds <= 0.0) return r.error = "window must be greater
↪ than zero", r;
54
55     const double hours = in.window_seconds / 3600.0;
56     const double compute_cost = in.price_per_instance_hour * in.instances *
↪ hours;
57     r.components.push_back({"compute", compute_cost, true});
58
59     if (in.price_per_gb_month_storage >= 0.0 && in.storage_gb >= 0.0) {
60         const double months = in.window_seconds / (730.0 * 3600.0); // 730 h
↪ billing month
61         r.components.push_back(
62             {"storage", in.price_per_gb_month_storage * in.storage_gb *
↪ months, true});

```

```

63     } else {
64         r.components.push_back({"storage", 0.0, false});
65     }
66
67     if (in.price_per_gb_egress >= 0.0 && in.egress_gb >= 0.0) {
68         r.components.push_back({"egress", in.price_per_gb_egress * in.
↪ egress_gb, true});
69     } else {
70         r.components.push_back({"egress", 0.0, false});
71     }
72
73     for (const auto& c : r.components) r.total_cost += c.amount;
74     r.units_of_work = in.throughput_units_per_second * in.window_seconds;
75     r.cost_per_unit = r.total_cost / r.units_of_work;
76     r.cost_per_million_units = r.cost_per_unit * 1e6;
77     r.ok = true;
78     return r;
79 }
80
81 } // namespace stratus::cost

```

Листинг 3: include/stratus/cost.hpp

Приложение Г. Контейнерен образ и локален стек

```

1 # Multi-stage build.
2 #
3 # The build stage carries a compiler, CMake and Ninja; the final stage
↪ carries nothing at
4 # all. Linking statically against musl makes that possible: the runtime image
↪ is the four
5 # executables and nothing else, so there is no package manager, no shell and
↪ no library to
6 # keep patched. The cost is that the image has no tools for poking at a
↪ running container,
7 # which is a fair trade for a workload whose only interface is three HTTP
↪ endpoints.
8 #

```

```

9 # The unit tests run inside the build stage. An image that compiled but fails
  ↳ its own
10 # tests should not exist.
11
12 FROM alpine:3.20 AS build
13 # build-base brings gcc, g++, musl-dev and make. Make rather than Ninja on
  ↳ purpose: it is
14 # already in build-base, and one fewer package is one fewer thing that can
  ↳ move.
15 RUN apk add --no-cache build-base cmake
16 WORKDIR /src
17
18 COPY CMakeLists.txt ./
19 COPY include ./include
20 COPY src ./src
21 COPY tests ./tests
22
23 RUN cmake -S . -B build \
24     -DCMAKE_BUILD_TYPE=Release \
25     -DCMAKE_EXE_LINKER_FLAGS="-static" \
26     && cmake --build build -j "$(nproc)" \
27     && ctest --test-dir build --output-on-failure \
28     && strip build/stratus-worker build/stratus-proxy build/stratus-loadgen \
29         build/stratus-autoscaler build/stratus-cost
30
31 FROM scratch
32 COPY --from=build /src/build/stratus-worker      /stratus-worker
33 COPY --from=build /src/build/stratus-proxy      /stratus-proxy
34 COPY --from=build /src/build/stratus-loadgen    /stratus-loadgen
35 COPY --from=build /src/build/stratus-autoscaler /stratus-autoscaler
36 COPY --from=build /src/build/stratus-cost       /stratus-cost
37
38 EXPOSE 8080 8081
39
40 # CMD, not ENTRYPOINT. One image carries all five executables and the compose
  ↳ file selects
41 # one per service with 'command: '; an ENTRYPOINT would turn that selection
  ↳ into arguments
42 # passed to the worker, and every service would silently start as a worker.

```

```
43 CMD ["/stratus-worker"]
```

ЛИСТИНГ 4: Dockerfile

```
1 # The local stack: N workers, a round robin reverse proxy in front, and a
  ↪ scraper.
2 #
3 # The worker service publishes no port and has no container_name, which is
  ↪ what makes
4 # 'docker compose up --scale worker=N' work: replicas differ only in their
  ↪ address, and
5 # the proxy finds them by resolving the service name.
6 #
7 # Each replica is pinned to one CPU and runs a single request thread. That is
  ↪ the whole
8 # point of the arrangement: capacity is then exactly one thread per replica,
  ↪ so fleet
9 # utilisation is a number the controller can compute rather than guess.
10
11 name: stratus
12
13 services:
14   worker:
15     build:
16       context: .
17       dockerfile: Dockerfile
18     image: stratus:local
19     command: ["/stratus-worker"]
20     environment:
21       STRATUS_PORT: "8081"
22       STRATUS_THREADS: "1"
23       STRATUS_DEFAULT_SIZE: "384"
24       STRATUS_DEFAULT_ITER: "500"
25     expose:
26       - "8081"
27     deploy:
28       resources:
29         limits:
```



```

30     cpus: "1.0"
31     memory: 128M
32     restart: unless-stopped
33
34 proxy:
35     build:
36         context: .
37         dockerfile: Dockerfile
38     image: stratus:local
39     command: ["/stratus-proxy"]
40     environment:
41         STRATUS_PORT: "8080"
42         STRATUS_BACKEND_HOST: "worker"
43         STRATUS_BACKEND_PORT: "8081"
44         STRATUS_THREADS: "64"
45         # Rediscovery period. Short enough that a new replica takes traffic
↪ within a couple
46         # of seconds of starting, long enough not to resolve on every single
↪ request.
47         STRATUS_DISCOVERY_TTL_MS: "1000"
48     ports:
49         - "8080:8080"
50     depends_on:
51         - worker
52     restart: unless-stopped
53
54     # The in-stack metrics scraper. It is the controller binary in observe only
↪ mode: it
55     # polls the aggregated /metrics on a fixed period and writes the fleet
↪ timeline, but
56     # never calls docker. The scaling controller itself runs on the host,
↪ because scaling a
57     # compose project from inside one of its own containers would require
↪ handing that
58     # container the docker socket.
59     scraper:
60         build:
61             context: .
62             dockerfile: Dockerfile

```

```

63 image: stratus:local
64 command:
65   - "/stratus-autoscaler"
66   - "--host"
67   - "proxy"
68   - "--port"
69   - "8080"
70   - "--interval"
71   - "2"
72   - "--dry-run"
73   - "--csv"
74   - "/out/fleet-metrics.csv"
75 depends_on:
76   - proxy
77 volumes:
78   - ./results:/out
79 restart: unless-stopped

```

Листинг 5: docker-compose.yml

Приложение Д. Описание на инфраструктурата

Общата част: обектът, който описва услугата, и изборът на модул. Двата модула за доставчици са в `infra/modules/` и не са включени тук заради обема им, 297 реда за AWS и 178 реда за Azure.

```

1 # The shared service definition.
2 #
3 # This object is the whole interface. Both provider modules accept exactly
4 ↪ this type and
5 # nothing else, so the service is described once and the choice of provider
6 ↪ is the value
7 # of a single variable. Anything a module needs that is not in here is, by
8 ↪ construction,
9 # a provider detail and belongs inside that module.
10
11 variable "target_provider" {
12   description = "Which provider module to instantiate: aws or azure."
13 }

```

```

10     type          = string
11
12     validation {
13         condition      = contains(["aws", "azure"], var.target_provider)
14         error_message = "target_provider must be aws or azure."
15     }
16 }
17
18 variable "region" {
19     description = "Provider region. The value is provider specific, the
↪ variable is not."
20     type          = string
21 }
22
23 variable "service" {
24     description = "The provider independent description of the Stratus service.
↪ "
25
26     type = object({
27         name          = string
28         image          = string
29         container_port = number
30
31         # CPU is given in Fargate CPU units, where 1024 units is one vCPU. The
↪ Azure module
32         # divides by 1024 to get the fractional core count Container Apps expects
↪ . A single
33         # unit was needed and this one at least has a defined conversion in both
↪ directions.
34         cpu_units = number
35         memory_mb = number
36
37         min_replicas      = number
38         max_replicas      = number
39         target_utilisation = number
40
41         env = map(string)
42         tags = map(string)
43     })

```

```

44
45     validation {
46         condition      = var.service.min_replicas >= 1 && var.service.max_replicas
↪ >= var.service.min_replicas
47         error_message = "min_replicas must be at least 1 and no greater than
↪ max_replicas."
48     }
49
50     validation {
51         condition      = var.service.target_utilisation > 0 && var.service.
↪ target_utilisation <= 1
52         error_message = "target_utilisation is a fraction in (0, 1]."
53     }
54 }

```

ЛИСТИНГ 6: infra/variables.tf

```

1  # Root configuration.
2  #
3  # Exactly one of the two modules is instantiated, chosen by target_provider.
↪ Both receive
4  # the same object and both return the same outputs, so switching providers is
↪ a change of
5  # one variable and nothing else. The count expressions are the only place in
↪ the whole
6  # configuration where the provider is named outside its own module.
7
8  module "aws" {
9      count  = var.target_provider == "aws" ? 1 : 0
10     source = "./modules/aws"
11
12     region = var.region
13     service = var.service
14 }
15
16 module "azure" {
17     count  = var.target_provider == "azure" ? 1 : 0
18     source = "./modules/azure"

```

```

19
20     region = var.region
21     service = var.service
22 }

```

Листинг 7: infra/main.tf

```

1  # The outputs are the second half of the shared interface. The load generator
   ↪ and the
2  # controller are pointed at ingress_url and never learn which provider
   ↪ answered.
3
4  output "ingress_url" {
5      description = "Base URL of the deployed service."
6      value = var.target_provider == "aws" ? one(module.aws[*].ingress_url) : one
   ↪ (module.azure[*].ingress_url)
7  }
8
9  output "object_store" {
10     description = "Name of the object store created for the deployment."
11     value = var.target_provider == "aws" ? one(module.aws[*].object_store) :
   ↪ one(module.azure[*].object_store)
12 }
13
14 output "platform_id" {
15     description = "Identifier of the compute platform holding the service."
16     value = var.target_provider == "aws" ? one(module.aws[*].platform_id) : one
   ↪ (module.azure[*].platform_id)
17 }
18
19 output "workload_identity" {
20     description = "Name of the identity the workload assumes to reach the
   ↪ object store."
21     value = var.target_provider == "aws" ? one(module.aws[*].workload_identity)
   ↪ : one(module.azure[*].workload_identity)
22 }

```

Листинг 8: infra/outputs.tf

Приложение Е. Генератор на натоварване

Генераторът е на C++ и е част от същото дърво на изграждане. Параметрите, с които е изпълняван при всяко измерване, са записани в `bench.ps1` и `demo.ps1` и са възпроизведени в Раздел V.

```
1 // stratus-loadgen: synthetic load with configurable concurrency and rate.
2 //
3 // Two modes, because they answer different questions. With --rate 0 the
4 // closed loop: N threads each send the next request as soon as the previous
5 // which measures how much work the service can absorb. With --rate R it is
6 // send schedule is fixed in advance, so a service that slows down
7 // delay in the reported latency instead of quietly throttling the client.
8 // is the one that shows what the autoscaler is for.
9 #include <algorithm>
10 #include <atomic>
11 #include <chrono>
12 #include <cstdio>
13 #include <fstream>
14 #include <iostream>
15 #include <mutex>
16 #include <string>
17 #include <thread>
18 #include <vector>
19
20 #include "stratus/args.hpp"
21 #include "stratus/http.hpp"
22 #include "stratus/net.hpp"
23 #include "stratus/stats.hpp"
24
25 namespace {
26
27 using Clock = std::chrono::steady_clock;
```

```

28
29 struct Sample {
30     double t_offset_s = 0.0;
31     double latency_s = 0.0;
32     bool ok = false;
33 };
34
35 void usage() {
36     std::cout <<
37         "stratus-loadgen --host H --port P [options]\n"
38         "  --path S      request target (default /work)\n"
39         "  --size N      work size parameter appended to the target (
↪ default 256)\n"
40         "  --iter N      iteration ceiling appended to the target (default
↪ 400)\n"
41         "  --concurrency N in flight requests (default 8)\n"
42         "  --rate R      target requests per second, 0 for closed loop (
↪ default 0)\n"
43         "  --duration S  measurement seconds (default 20)\n"
44         "  --warmup S    discarded seconds before measuring (default 3)\n"
45         "  --csv FILE    write per request samples\n";
46 }
47
48 } // namespace
49
50 int main(int argc, char** argv) {
51     stratus::args::Args a(argc, argv);
52     if (a.has("help")) {
53         usage();
54         return 0;
55     }
56
57     const std::string host = a.str_or("host", "127.0.0.1");
58     const auto port = static_cast<std::uint16_t>(a.int_or("port", 8080));
59     const std::string path = a.str_or("path", "/work");
60     const int size = a.int_or("size", 256);
61     const int iter = a.int_or("iter", 400);
62     const int concurrency = std::max(1, a.int_or("concurrency", 8));
63     const double rate = a.num_or("rate", 0.0);

```

```

64     const double duration = a.num_or("duration", 20.0);
65     const double warmup = a.num_or("warmup", 3.0);
66     const std::string csv = a.str_or("csv", "");
67
68     const std::string target = path + "?size=" + std::to_string(size) +
69                                     "&iter=" + std::to_string(iter);
70
71     stratus::net::startup();
72
73     std::vector<std::vector<Sample>> per_thread(static_cast<std::size_t>(
↪ concurrency));
74     std::atomic<bool> running{true};
75     std::atomic<long long> issued{0};
76
77     const auto t_start = Clock::now();
78     const auto t_end = t_start + std::chrono::duration_cast<Clock::duration>(
79                                     std::chrono::duration<double>(warmup +
↪ duration));
80
81     auto worker = [&](int idx) {
82         auto& out = per_thread[static_cast<std::size_t>(idx)];
83         out.reserve(4096);
84         while (running.load(std::memory_order_relaxed)) {
85             if (rate > 0.0) {
86                 // Open loop. The nth request is due at n / rate from the
↪ start, computed
87                 // from a shared counter so the whole client obeys one
↪ schedule.
88                 const long long n = issued.fetch_add(1, std::
↪ memory_order_relaxed);
89                 const auto due = t_start + std::chrono::duration_cast<Clock::
↪ duration>(
90                                     std::chrono::duration<double>(
91                                     static_cast<double>(n) /
↪ rate));
92                 if (due > t_end) break;
93                 std::this_thread::sleep_until(due);
94             }
95             const auto t0 = Clock::now();

```



```

96         if (t0 >= t_end) break;
97         auto resp = stratus::net::get(host, port, target, 60000);
98         const auto t1 = Clock::now();
99
100        Sample s;
101        s.t_offset_s = std::chrono::duration<double>(t0 - t_start).count
↵ ();
102        s.latency_s = std::chrono::duration<double>(t1 - t0).count();
103        s.ok = resp.has_value() && resp->status == 200;
104        out.push_back(s);
105    }
106};
107
108std::vector<std::thread> pool;
109pool.reserve(static_cast<std::size_t>(concurrency));
110for (int i = 0; i < concurrency; ++i) pool.emplace_back(worker, i);
111
112std::this_thread::sleep_until(t_end);
113running.store(false);
114for (auto& t : pool) t.join();
115
116// Discard the warmup window: the first requests pay for cold caches, for
↵ lazily
117// created connections and, when the stack was just started, for
↵ containers that are
118// still coming up. Including them would report a system that does not
↵ exist after the
119// first three seconds.
120std::vector<Sample> all;
121for (auto& v : per_thread)
122    for (auto& s : v)
123        if (s.t_offset_s >= warmup) all.push_back(s);
124
125if (!csv.empty()) {
126    std::ofstream f(csv);
127    f << "offset_s,latency_s,ok\n";
128    for (const auto& s : all)
129        f << s.t_offset_s << ',' << s.latency_s << ',' << (s.ok ? 1 : 0)
↵ << '\n';

```

```

130     }
131
132     std::vector<double> ok_lat;
133     long long failed = 0;
134     for (const auto& s : all) {
135         if (s.ok) ok_lat.push_back(s.latency_s);
136         else ++failed;
137     }
138     std::sort(ok_lat.begin(), ok_lat.end());
139
140     const double measured = duration;
141     const double thr = static_cast<double>(ok_lat.size()) / (measured > 0 ?
↪ measured : 1.0);
142
143     auto ms = [](double s) { return s * 1000.0; };
144     std::printf("\n=== stratus-loadgen ===\n");
145     std::printf("target          http://%s:%u%s\n", host.c_str(),
↪ static_cast<unsigned>(port),
146                 target.c_str());
147     std::printf("concurrency      %d\n", concurrency);
148     std::printf("rate requested    %s\n", rate > 0.0 ? std::to_string(rate).
↪ c_str() : "closed loop");
149     std::printf("warmup / measure  %.1f s / %.1f s\n", warmup, measured);
150     std::printf("requests ok       %zu\n", ok_lat.size());
151     std::printf("requests failed   %lld\n", failed);
152     std::printf("throughput        %.2f req/s\n", thr);
153     std::printf("work rate         %.2f Miter/s\n",
154                 thr * static_cast<double>(size) * static_cast<double>(size) *
155                 static_cast<double>(iter) / 1e6);
156     std::printf("latency mean      %.2f ms\n", ms(stratus::stats::mean(ok_lat
↪ )));
157     std::printf("latency p50       %.2f ms\n", ms(stratus::stats::
↪ percentile_sorted(ok_lat, 0.50)));
158     std::printf("latency p90       %.2f ms\n", ms(stratus::stats::
↪ percentile_sorted(ok_lat, 0.90)));
159     std::printf("latency p95       %.2f ms\n", ms(stratus::stats::
↪ percentile_sorted(ok_lat, 0.95)));
160     std::printf("latency p99       %.2f ms\n", ms(stratus::stats::
↪ percentile_sorted(ok_lat, 0.99)));

```

```

161     std::printf("latency max      %.2f ms\n", ok_lat.empty() ? 0.0 : ms(
↪ ok_lat.back()));
162     if (!csv.empty()) std::printf("samples written  %s\n", csv.c_str());
163     std::printf("\n");
164
165     return ok_lat.empty() ? 1 : 0;
166 }

```

Листинг 9: src/loadgen_main.cpp

Приложение Ж. Сурови резултати от измерванията

Необработените изходни данни са в директорията `results/` на хранилището:

- `bench-throughput.csv`, таблицата от Раздел VI в машинен вид;
- `bench-throughput.txt`, пълните отчети на генератора за всяко повторение;
- `bench-latency-w{1,2,4,6}-run{1,2,3}.csv`, по един ред за всяка отделна заявка, с отместване във времето, закъснение и изход;
- `bench-reaction.csv`, времената за реакция при увеличаване;
- `scaling-timeline.csv` и `autoscaler.log`, решенията на контролера;
- `latency.csv`, извадката от демонстрационното изпълнение;
- `fleet-metrics.csv`, записът на събирача вътре в стека.

Обработката в Раздел VI се свежда до медиана по три повторения и може да бъде проверена независимо от тези файлове.

Приложение З. Екранни форми

[TODO: Да се приложат екранни изображения от таблото за наблюдение. Табло не е изграждано: метриките се четат от крайната точка `/metrics` и от файловете в `results/`, а изграждането на табло не добавя измерване, а само изглед към същите данни.]